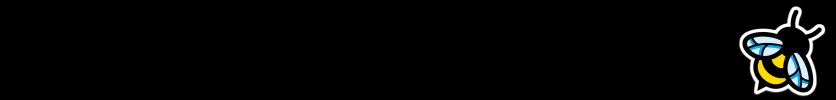
Whole-System Instrumentation in Kubernetes: High-Performance Observability with eBPF



Roger Coll Aumatell

DevBcn 2026

elastic

> What is OpenTelemetry?

```
- java -javaagent:elastic-apm-agent.jar
+ java -javaagent:opentelemetry-javaagent.jar
```

```
-   -Delastic.apm.server_url=http://127.0.0.1:8200
+   -Dotel.exporter.otlp.endpoint=http://127.0.0.1:9200
```

Metrics

Logs

Traces

What's happening in our systems?

{ http.error.rate = 75%, timestamp = … }

Why is it happening?

{ msg = “ad not found”, timestamp = … }

Where is it happening?

```
{ name = “get_user”, span_id = …, parent_id = … }
```

```java
private static final Meter meter = GlobalOpenTelemetry.getMeter("ad");
private static final LongCounter httpRequestsCounter =
      meter
          .counterBuilder("http.server.active_requests")
          .setDescription("Counts the server HTTP active requests")
          .build();

httpRequestsCounter.add(1);
```

```java
private static final Logger logger = LogManager.getLogger(AdService.class);
logger.log(Level.WARN, "GetAds Failed with status {}", e.getStatus());
```

```java
private static final Tracer tracer = GlobalOpenTelemetry.getTracer("ad");
Span span = tracer.spanBuilder("getRandomAds").startSpan();
// work
span.end();
```

> Did you instrument 100% of the code execution path?

> Hot code paths regressions?

> Why is the kernel on-CPU so much?

> Are the CPUs stalled on memory I/O?

What we don’t know?

Unknown unknowns

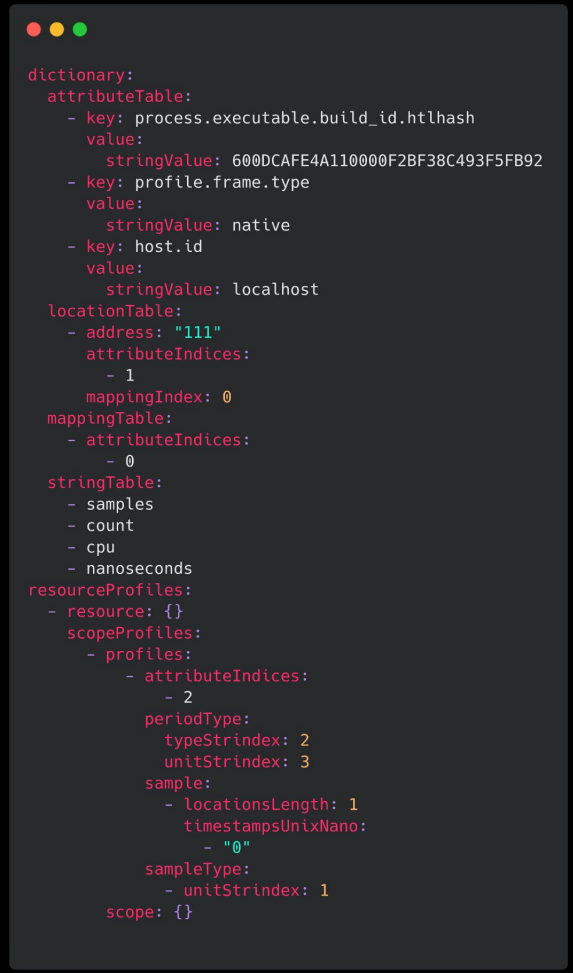
Profiles

What are profiles?

```
30      db_select;get_balance;serve_requests;main
20      get_balance;serve_requests;main
5       report_to_irs;main
10      make_widthdraw;serve_requests;main
2       db_insert;make_widthdraw;serve_requests;main
```

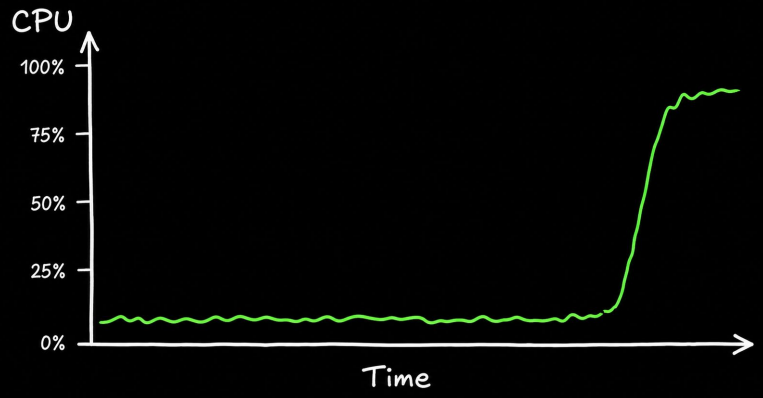
OpenTelemetry Profiles

alpha

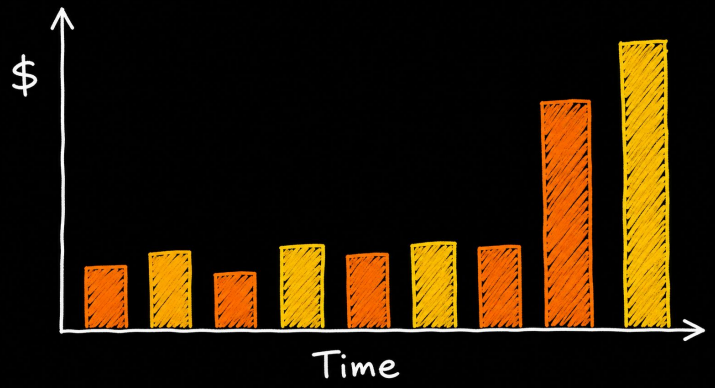


```yaml
dictionary:
  attributeTable:
    - key: process.executable.build_id.htlhash
      value:
        stringValue: 600DCAFE4A110000F2BF38C493F5FB92
    - key: profile.frame.type
      value:
        stringValue: native
    - key: host.id
      value:
        stringValue: localhost
  locationTable:
    - address: "111"
      attributeIndices:
        - 1
      mappingIndex: 0
  mappingTable:
    - attributeIndices:
        - 0
  stringTable:
    - samples
    - count
    - cpu
    - nanoseconds
resourceProfiles:
  - resource: {}
    scopeProfiles:
      - profiles:
          - attributeIndices:
              - 2
            periodType:
              typeStrindex: 2
              unitStrindex: 3
            sample:
              - locationsLength: 1
                timestampsUnixNano:
                  - "0"
            sampleType:
              - unitStrindex: 1
        scope: {}
```

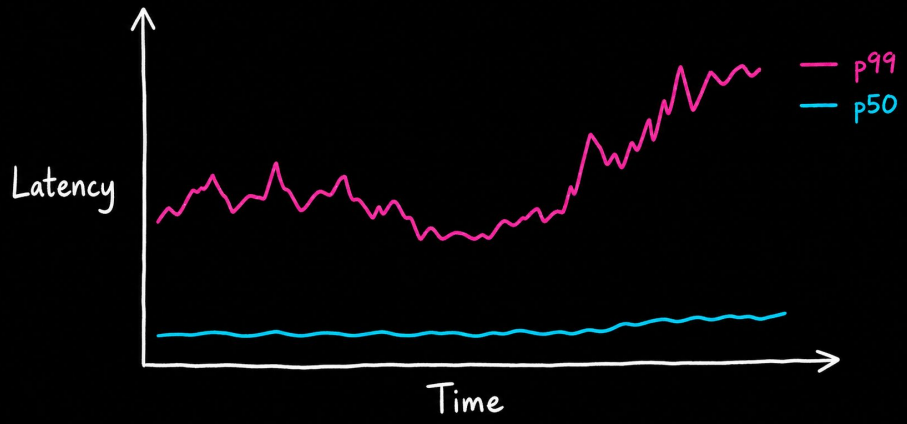
Incidents



Costs



Performance



OpenTelemetry Profiling Receiver

Frictionless deployment, no code changes or app restarts.

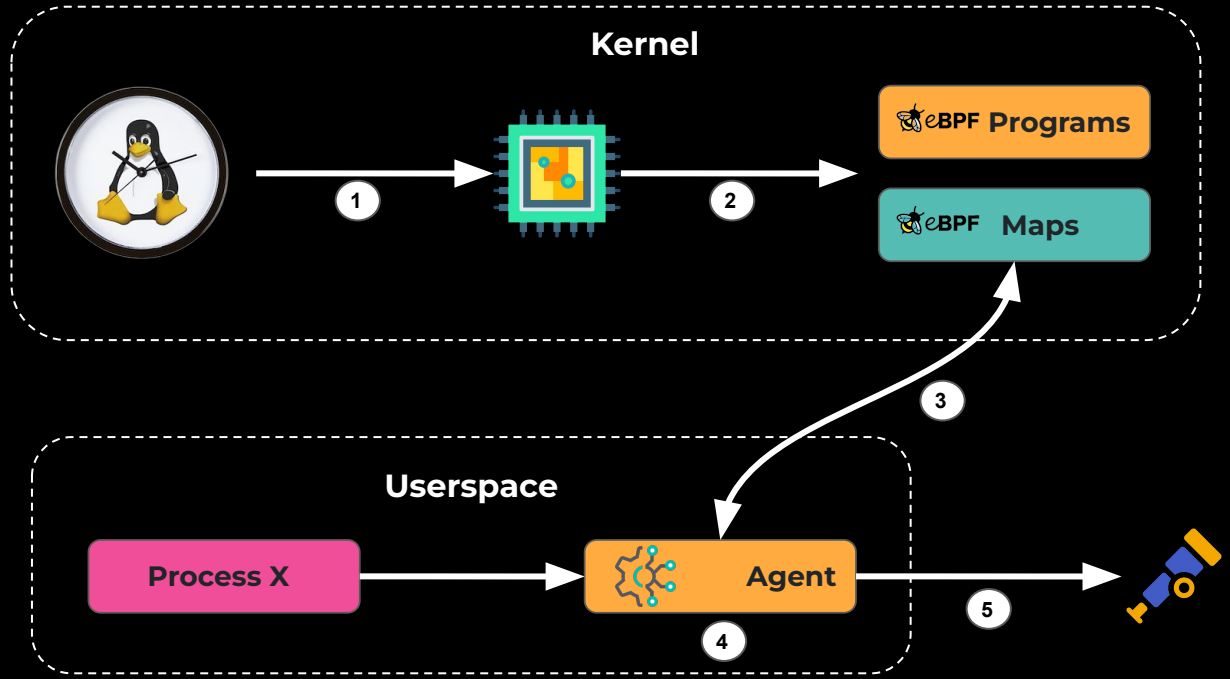
Whole-system visibility: from the Kernel through user space.

Multiple High Level Language runtimes.

Extremely Low Overhead: aiming for: <1% system CPU, ~250MB of RAM

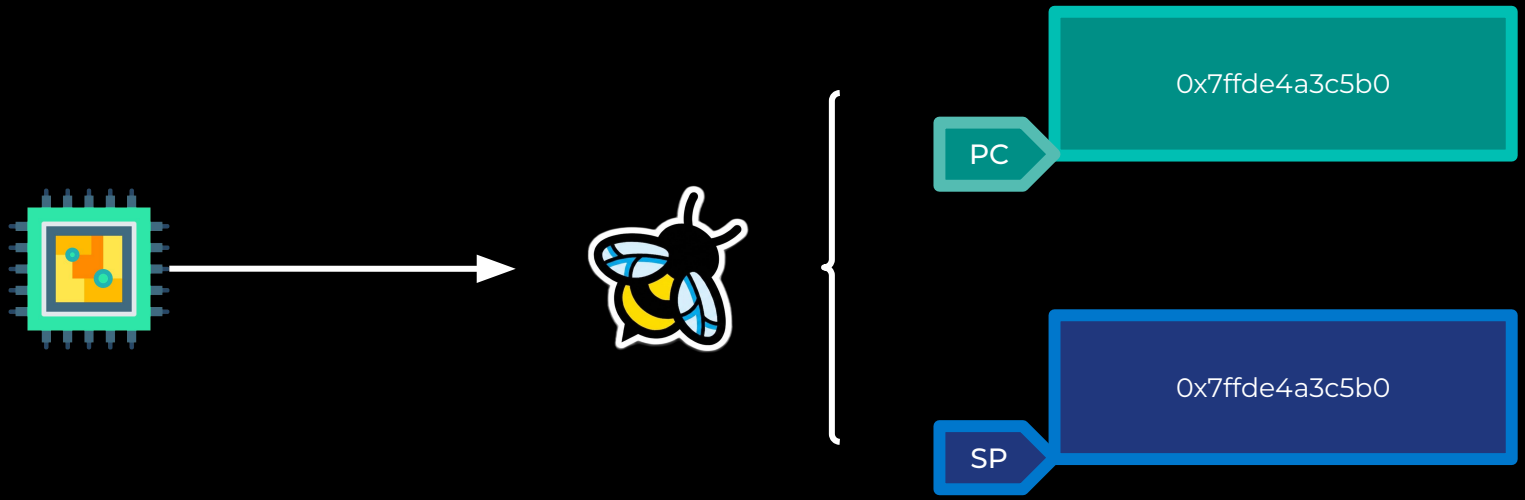
Elastic donates profiling agent to OpenTelemetry

> What is eBPF?

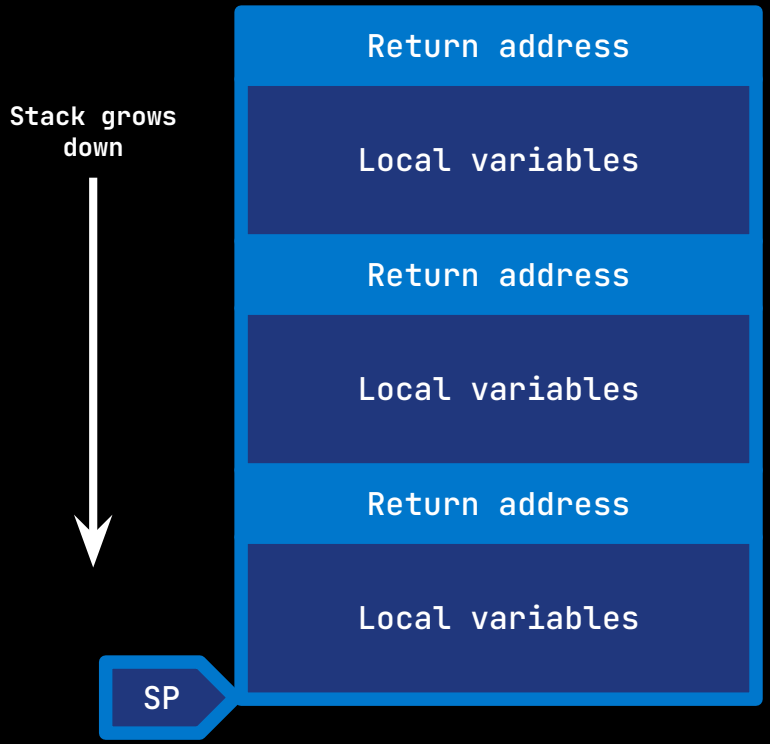


1. CPU interrupt (19Hz)
2. Stack unwinding
3. Process-specific data
4. Symbolization
5. OTLP Profiles

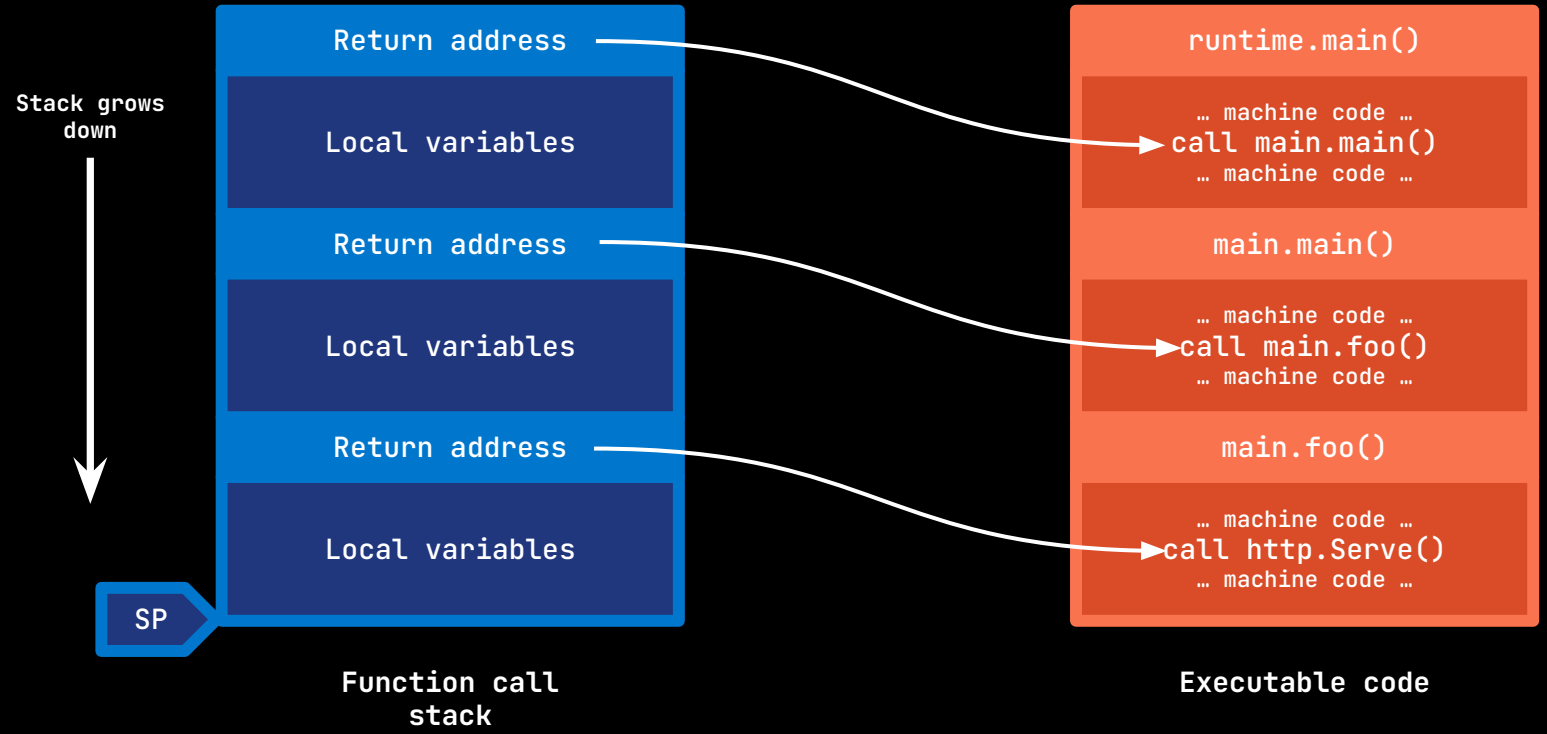
Stack unwinding (FPO)



Stack unwinding (FPO)



Stack unwinding (FPO)



Stack unwinding

Native vs High Level Languages

Native executables:

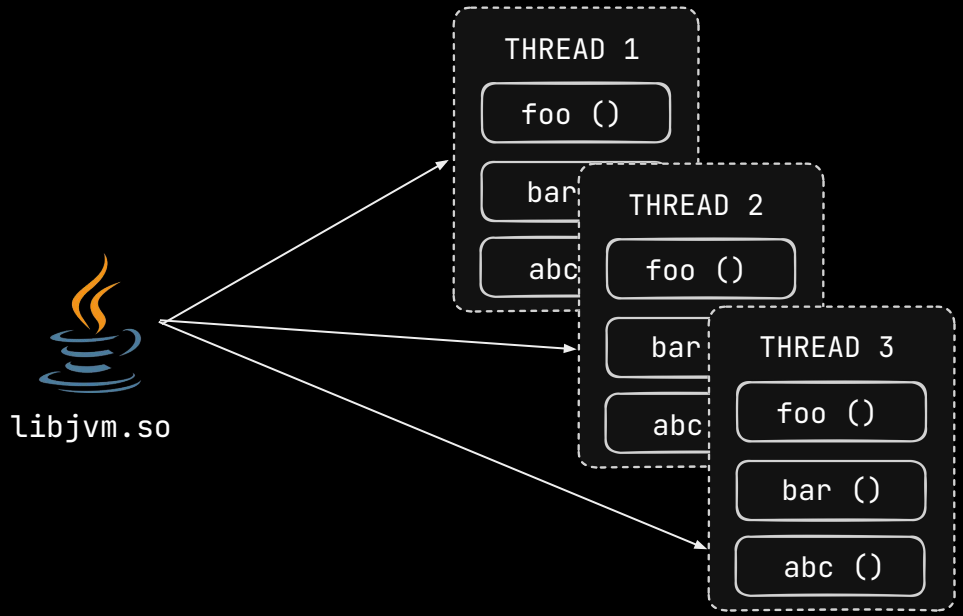
- C++, Rust, C, etc. → via ELFs metadata (.eh_frame)
- Go relies on the binary *.gopclntab* section.

```
String dump of section '.gopclntab':
  ...
  [19140f]  github.com/collector/receiver/xreceiver.NewFactory
  [19144b]  github.com/collector/receiver.NewFactory
  [19147d]  github.com/collector/receiver/xreceiver.factory.CreateDefaultConfig
```

Stack unwinding

Native vs High Level Languages

HLL runtimes: Every language needs to keep track of a call stack.




eprofiler-tui 100.0%
eprofiler-tui+0x000000000056f88d [Native] 86.8% entry_SYS12.1%…
eprofiler-tui+0x00000000004729f6 [Native] 86.8% do_syscal12.1%…
eprofiler-tui+0x0000000000eef095 [Native] 86.8% ksys_writ…
eprofiler-tui+0x0000000000472a13 [Native] 86.8% vfs_write…
eprofiler-tui+0x0000000000498354 [Native] 86.8% file_tty_…
eprofiler-tui+0x000000000044eb11 [Native] 86.8% n_tty_wr…
eprofiler-tui+0x000000000056d963 [Native] 86.8% pty_w…
eprofiler-tui+0x000000000059e1e5 [Native] 86.8% tty_i…
eprofiler-tui+0x000000000057 c09 [Native] 86.8% que…
eprofiler-tui+0x0000000000573600 [Native] 45.1%
eprofiler-tui+0x0000000000572c32 [Native] 45.1%
eprofiler-tui+0x0000000000535372 [Native] 45.1%
eprofiler-tui+0x00000000005736b3 [Native] 41.8%
eprofiler-tui+0x0000000000573112 [Native] 41.8%
eprofiler-tui+0x000000000048cb05 [Na 34.1%
eprofil 11.0%…
eprofile…
eprof…
epro…
epr…
ep…
eprofiler-…
eprofi…
eprofil…
epro…
eprof…


eprofiler-tui
100.0%
main [Native]
89.6%
std::rt::lang_start [Native]
89.6%
std::rt::lang_start_internal [Native] / std::panic::catch_unwind [Native] [Inline] / std::panicking::catch_unwi
89.6%
std::rt::lang_start::{{closure}} [Native]
89.6%
std::sys::backtrace::__rust_begin_short_backtrace [Native]
89.6%
core::ops::function::FnOnce::call_once [Native]
89.6%
eprofiler_tui::main [Native]
89.6%
eprofiler_tui::tui::Tui::draw [Native]
88.1%
ratatui_core::terminal::terminal::Terminal<B>::draw [Native]
88.1%
ratatui_core::terminal::terminal::Terminal<B>::try_draw [Native]
88.1%
ratatui_core::terminal::terminal::Terminal<B>::draw::{{cl
48.6%
ratatui_core::terminal::terminal::Termina
37.4%
eprofiler_tui::tui::Tui::draw::{{closure}} [Native]
48.6%
ratatui_core::buffer::buffer::Buffer::
35.3%
eprofiler_tui::tui::ui::render [Native]
48.6%
<str as unicode_
18.0%
eprofiler_tui::tui::ui::render_flamegraph [Na
40.3%
unicode_width::t
18.0%
eprofiler_tu
15.1%
core::iter::t
16.2%
entry_
10.1%
do_sys
10.1%
ksys_writ
vfs_writ
file_tty
n_tty
pt
tt
qu

Symbolization

Stack trace offsets

```
moduleID1+0×6c8
moduleID1+0×9c9
moduleID2+0×c8714
moduleID2+0×281e1
```

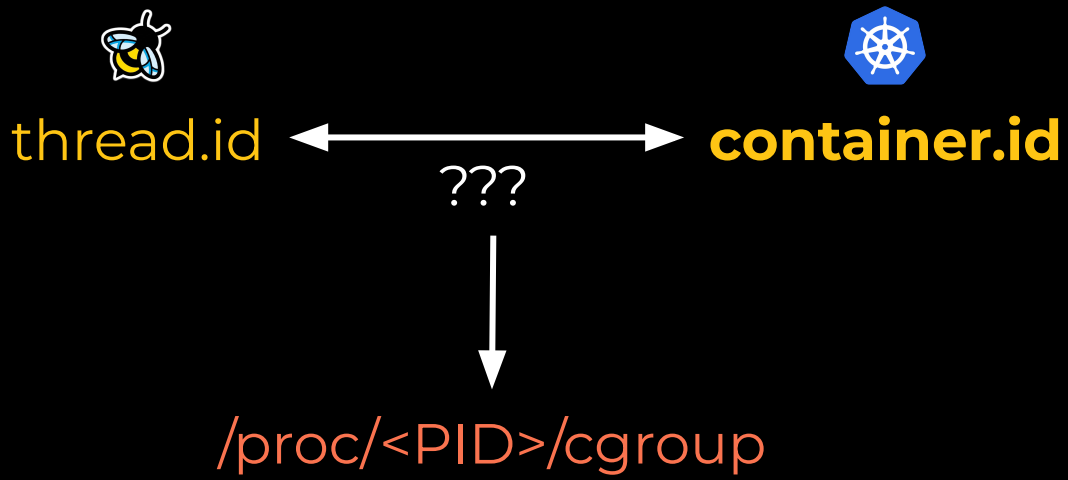
Human-readable symbols

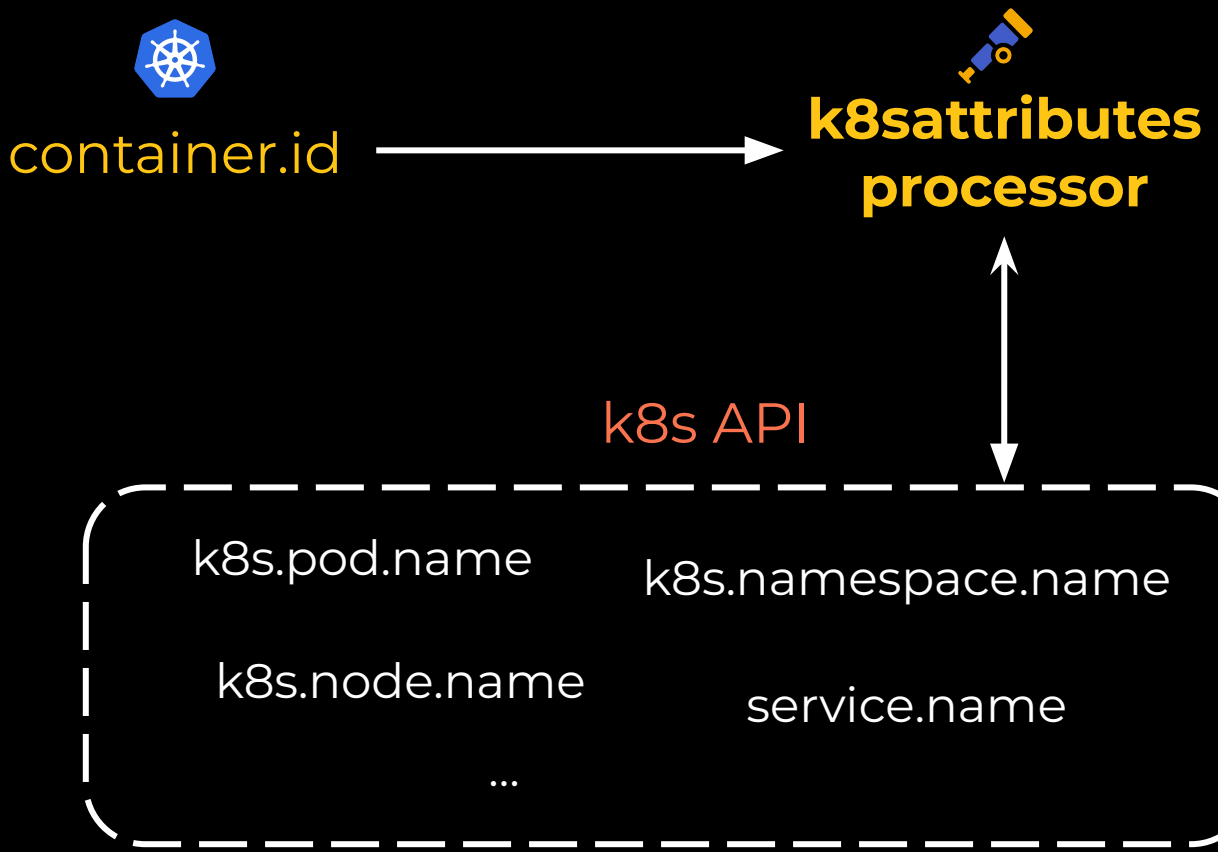
```
V8::EntryFrame [JS]
V8::StubFrame [JS]
getPlaywrightVersion in /usr/local/..
V8::ExitFrame [JS]
```

Supported Languages

*DotNet is not supported on ARM64

Standalone Demo


thread.id
container.id
???
/proc/<PID>/cgroup


container.id
k8sattributes processor
k8s API
k8s.pod.name
k8s.namespace.name
k8s.node.name
service.name
...

K8s OTel Collector

Demo

https://github.com/open-telemetry/opentelemetry-ebpf-profiler

https://github.com/rogercoll/eprofiler-tui

https://github.com/open-telemetry/opentelemetry-specification

https://www.elastic.co/observability-labs/blog/otel-profiling-metrics

Q & A

Gràcies